Table of Contents

Report on Demonstrating Insecure JWT Configuration Vulnerability

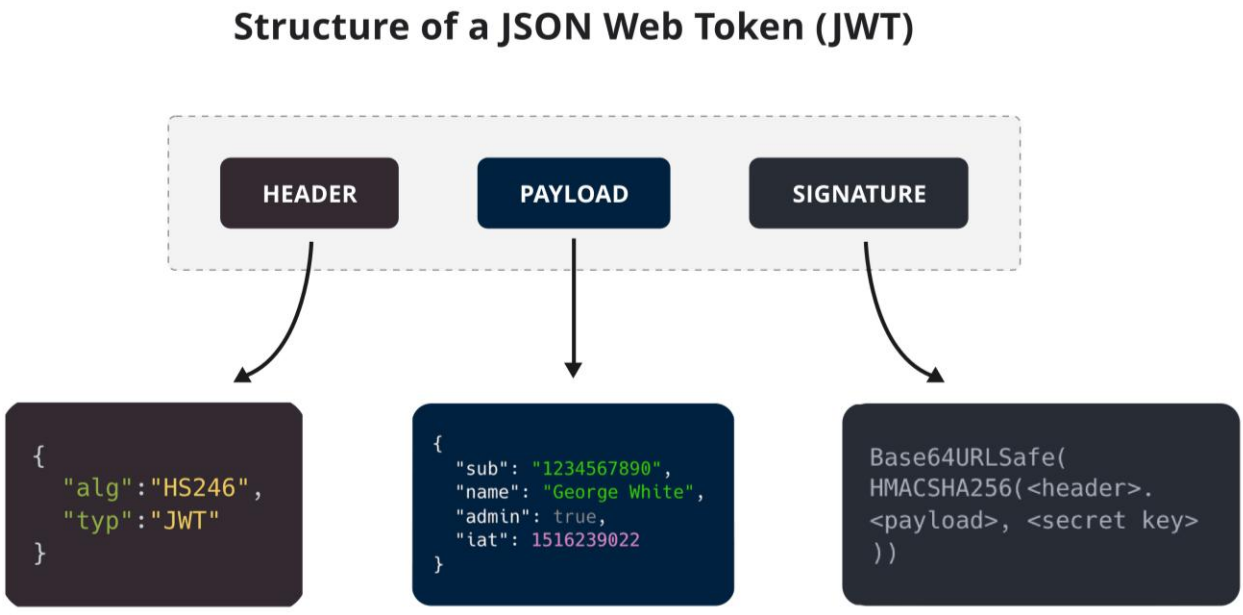
Task Overview

The primary objective of this task was to develop a login page that uses JSON Web Tokens (JWTs) for authentication. The specific requirement was to deliberately configure JWTs insecurely, such that when the JWT token is decoded, the password can be revealed.

JWTs or JSON Web Tokens are most commonly used to identify an authenticated user. They are issued by an authentication server and are consumed by the client-server (to secure its APIs). JWT (JSON Web Token) is a compact, URL-safe means of representing claims to be transferred between two parties. It consists of three parts: Header, Payload, and Signature. The payload of a JWT is base64 encoded, not encrypted. This means anyone with the token can decode and read the information, making it insecure to include sensitive data.

A JWT contains three parts:

- **Header**: Consists of two parts:
    - The signing algorithm that's being used.
    - The type of token, which, in this case, is mostly "JWT".
- **Payload**: The payload contains the claims or the JSON object.
- **Signature**: A string that is generated via a cryptographic algorithm that can be used to verify the integrity of the JSON payload.



Potential risks of insecure JWT configurations: Exposure of sensitive information, token interception, unauthorized access, and increased vulnerability to various attacks like replay attacks.

This demonstrates a critical security vulnerability related to improper JWT configuration.

Technologies Used

- **Python**: The primary programming language for the backend.
- **Flask**: A lightweight web framework used to develop the web application.
- **JWT (PyJWT library)**: Used for encoding and decoding JSON Web Tokens.
- **HTML/CSS/JavaScript**: For creating and managing the frontend of the application.
- **VS Code**: The Integrated Development Environment (IDE) used for coding.

Steps Undertaken

1. **Setting Up the Flask Application**:
    - Installed Flask using pip install flask.
    - Created a basic Flask app structure with app.py as the main application file.
2. **Generating JWT Tokens**:
    - Implemented a function to generate JWT tokens that include the username and password in the payload.
    - Used the PyJWT library to handle the encoding and decoding of JWT tokens.
3. **Creating the Login Endpoint**:
    - Set up a /login route to handle POST requests with username and password.
    - Validated credentials and generated a JWT token upon successful login.
4. **Frontend Development**:
    - Created an index.html file with a simple login form.
    - Used Bootstrap for styling and better user interface design.
    - Added JavaScript to handle form submission, display the JWT token, and decode the token to show the payload.
5. **Handling Favicon Requests**:
    - Added a placeholder favicon.ico to avoid 404 errors for favicon requests.
6. **Running the Application**:
    - Used VS Code to write and run the code.
    - Executed the Flask application using the command python app.py.

Code Overview

Backend (app.py)

- Configured Flask with routes for serving the login page and handling login requests.
- Implemented JWT generation with a deliberately insecure configuration that includes the password in the token payload.

Frontend (index.html)

- Designed a login form using HTML and Bootstrap.
- Implemented JavaScript to handle the login process and JWT decoding, demonstrating the insecure configuration.

Detailed Process Description

1. **Login Page**: The login page allows users to input their username and password.
2. **JWT Token Generation**: Upon login, the server validates the credentials and generates a JWT token containing the username and password in its payload.
3. **Token Display and Decoding**: The token is displayed to the user, and JavaScript functions are provided to decode the token, revealing the password.

Security Implications

Vulnerability Demonstrated

- **Insecure JWT Configuration**: The inclusion of sensitive information (password) in the JWT payload, which can be easily decoded, represents a severe security flaw.

Mitigation Strategies

1. **Avoid Sensitive Data in JWT Payloads**: Never include sensitive information such as passwords in the JWT payload.
2. **Use Strong Secret Keys**: Ensure that the secret key used for signing JWTs is strong and securely stored.
3. **Implement Secure Token Expiry**: Set appropriate expiration times for JWT tokens to limit their validity.
4. **Use HTTPS**: Always transmit JWT tokens over secure channels (HTTPS) to prevent interception.
5. **Regularly Rotate Keys**: Implement key rotation strategies to minimize the impact of a compromised key.

Vulnerable App Code:

App.py:

```
from flask import Flask, request, jsonify, send_from_directory
```

```
import jwt
import datetime
import os

app = Flask(__name__)

# Secret key for JWT encoding/decoding
SECRET_KEY = 'my_super_secret_key'

def generate_jwt(username, password):
    """
    Generates a JWT token with the username and password in the payload.
    """
    payload = {
        'username': username,
        'password': password,
        'exp': datetime.datetime.utcnow() + datetime.timedelta(minutes=30)
    }
    token = jwt.encode(payload, SECRET_KEY, algorithm='HS256')
    return token

@app.route('/')
def index():
    """
    Serves the login HTML page.
    """
    return send_from_directory(os.path.join(app.root_path, 'static'), 'index.html')

@app.route('/login', methods=['POST'])
def login():
    """
    Handles login requests. Validates credentials and returns a JWT token if valid.
    """
    data = request.get_json()
    if not data or 'username' not in data or 'password' not in data:
        return jsonify({'message': 'Bad Request'}), 400

    username = data['username']
    password = data['password']

    # Example credentials for demonstration
    if username == 'admin123' and password == 'alisha1234567':
        token = generate_jwt(username, password)
        return jsonify({'token': token})
    else:
        return jsonify({'message': 'Invalid credentials'}), 401

if __name__ == '__main__':
```

```
    # Running the Flask application in debug mode
    app.run(debug=True)
```

Static (Folder):

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Page</title>
    <link href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"
rel="stylesheet">
    <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.16.0/umd/popper.min.js"></script>
    <script src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></script>
    <style>
        .token-box {
            word-wrap: break-word;
            white-space: pre-wrap;
            background-color: #d4edda;
            border-color: #c3e6cb;
            color: #155724;
            padding: 10px;
            border-radius: 5px;
            margin-top: 10px;
            position: relative;
        }
        .copy-btn, .decode-btn {
            position: absolute;
            top: 5px;
            cursor: pointer;
            color: #155724;
            background: none;
            border: none;
        }
        .copy-btn {
            right: 10px;
        }
        .decode-btn {
            right: 60px;
        }
        .spinner-border {
            display: none;
        }
        .toast {
            position: fixed;
            top: 20px;
            right: 20px;
```

```
        z-index: 1050;
      }
      .jwt-info {
        margin-top: 10px;
        display: none;
      }
      .decoded-step {
        margin-top: 10px;
        border: 1px solid #c3e6cb;
        border-radius: 5px;
        padding: 10px;
        background-color: #f8f9fa;
      }
    </style>
</head>
<body>
    <div class="container">
      <div class="row justify-content-center">
        <div class="col-md-6">
          <h2 class="text-center mt-5">Login Page</h2>
          <form id="loginForm" class="mt-3 needs-validation" novalidate>
            <div class="form-group">
              <label for="username">Username:</label>
              <input type="text" id="username" name="username" class="form-control" required>
              <div class="invalid-feedback">
                Please enter your username.
              </div>
            </div>
            <div class="form-group">
              <label for="password">Password:</label>
              <input type="password" id="password" name="password" class="form-control"
required>
              <div class="invalid-feedback">
                Please enter your password.
              </div>
            </div>
            <button type="submit" class="btn btn-primary btn-block">
              Login
              <span class="spinner-border spinner-border-sm" role="status" aria-
hidden="true"></span>
            </button>
          </form>
          <div id="response" class="mt-3"></div>
        </div>
      </div>
    </div>

    <div id="toast-container"></div>
```

```
<script>
  (function() {
    'use strict';
    window.addEventListener('load', function() {
      var forms = document.getElementsByClassName('needs-validation');
      Array.prototype.filter.call(forms, function(form) {
        form.addEventListener('submit', function(event) {
          if (form.checkValidity() === false) {
            event.preventDefault();
            event.stopPropagation();
          }
          form.classList.add('was-validated');
        }, false);
      });
    }, false);
  })();

  document.getElementById('loginForm').addEventListener('submit', async function(event) {
    event.preventDefault();

    const form = event.target;
    if (form.checkValidity() === false) {
      event.stopPropagation();
      form.classList.add('was-validated');
      return;
    }
    form.classList.remove('was-validated');

    const spinner = document.querySelector('.spinner-border');
    spinner.style.display = 'inline-block';

    const username = document.getElementById('username').value;
    const password = document.getElementById('password').value;

    const response = await fetch('/login', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ username, password })
    });

    const result = await response.json();

    spinner.style.display = 'none';

    const responseDiv = document.getElementById('response');
```

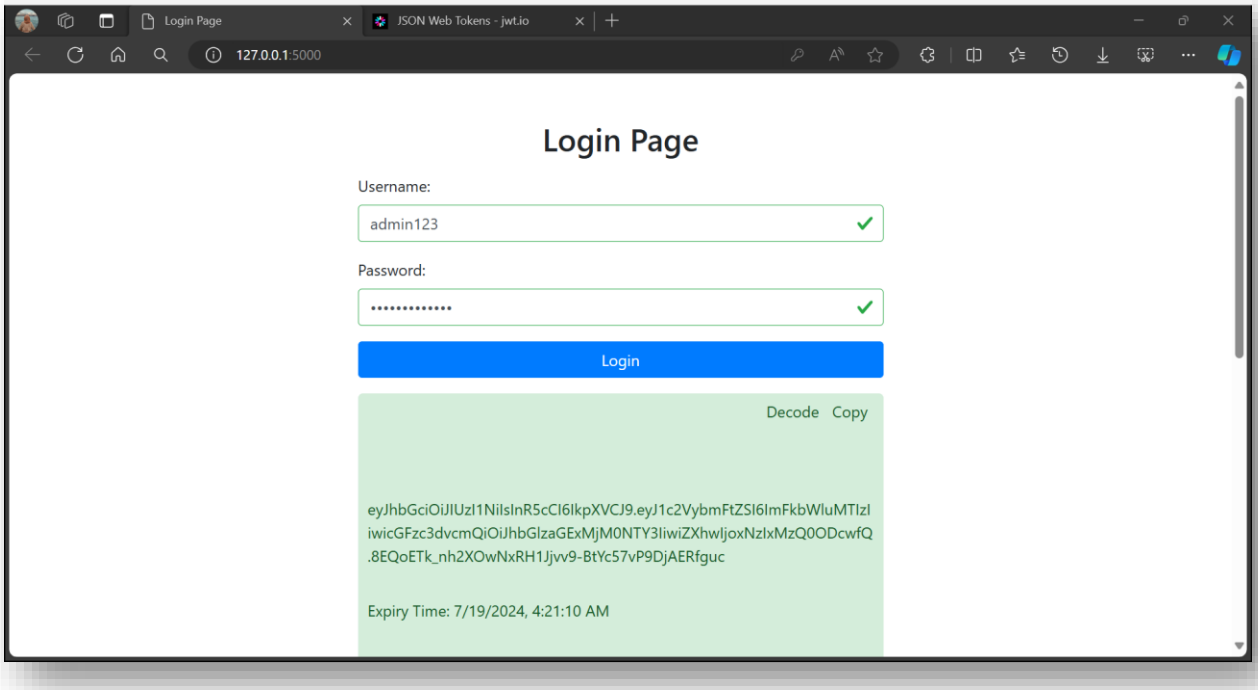
```
        if (response.ok) {
            const tokenPayload = JSON.parse(atob(result.token.split('.')[1]));
            const expiryTime = new Date(tokenPayload.exp * 1000).toLocaleString();

            responseDiv.innerHTML = `
                <div class="token-box">
                    <button class="copy-btn" onclick="copyToClipboard()" data-toggle="tooltip" data-
placement="top" title="Copy to clipboard">Copy</button>
                    <button class="decode-btn" onclick="decodeToken()" data-toggle="tooltip" data-
placement="top" title="Decode token">Decode</button>
                    <span id="jwtToken">${result.token}</span>
                    <div class="mt-2">Expiry Time: ${expiryTime}</div>
                    <div class="jwt-info" id="jwtInfo">
                        <strong>Decoded JWT:</strong>
                        <div class="decoded-step" id="headerStep"></div>
                        <div class="decoded-step" id="payloadStep"></div>
                        <div class="decoded-step" id="signatureStep"></div>
                    </div>
                </div>`;
            $('[data-toggle="tooltip"]').tooltip();
            showToast('Login successful! JWT token generated.', 'success');
        } else {
            responseDiv.innerHTML = '<div class="alert alert-danger">Error: ' + result.message + '</div>';
            showToast('Login failed. Please check your credentials.', 'danger');
        }
    });

    function copyToClipboard() {
        const token = document.getElementById('jwtToken').innerText;
        navigator.clipboard.writeText(token).then(function() {
            showToast('JWT Token copied to clipboard', 'success');
        }, function(err) {
            showToast('Could not copy text: ' + err, 'danger');
        });
    }

    function decodeToken() {
        const token = document.getElementById('jwtToken').innerText;
        const parts = token.split('.');

        const header = JSON.parse(atob(parts[0]));
        const payload = JSON.parse(atob(parts[1]));
        const signature = parts[2];

        document.getElementById('headerStep').innerText = `Header: ${JSON.stringify(header, null,
2)}`;
        document.getElementById('payloadStep').innerText = `Payload: ${JSON.stringify(payload, null,
2)}`;
```

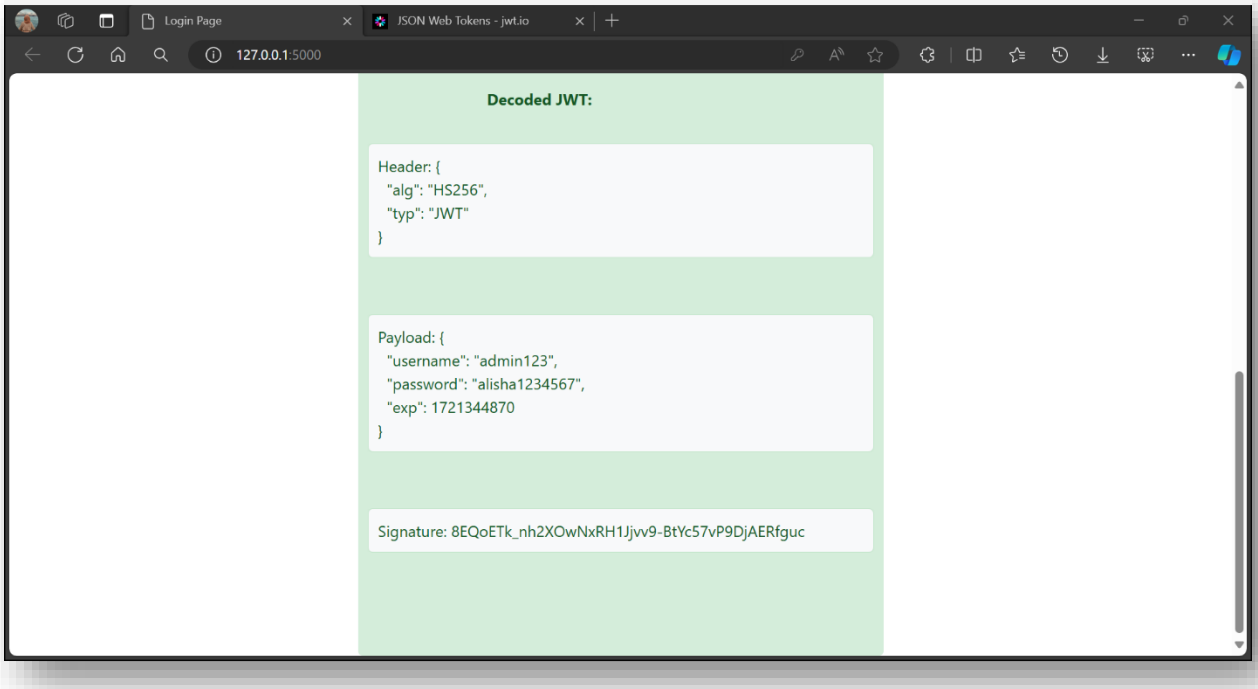
```
        document.getElementById('signatureStep').innerText = `Signature: ${signature}`;

        document.getElementById('jwtInfo').style.display = 'block';
    }

    function showToast(message, type) {
        const toastContainer = document.getElementById('toast-container');
        const toast = document.createElement('div');
        toast.className = `toast align-items-center text-white bg-${type} border-0`;
        toast.setAttribute('role', 'alert');
        toast.setAttribute('aria-live', 'assertive');
        toast.setAttribute('aria-atomic', 'true');

        toast.innerHTML = `
            <div class="d-flex">
                <div class="toast-body">
                    ${message}
                </div>
                <button type="button" class="ml-2 mb-1 close" data-dismiss="toast" aria-label="Close">
                    <span aria-hidden="true">&times;</span>
                </button>
            </div>`;

        toastContainer.appendChild(toast);
        $('.toast').toast({ delay: 3000 });
        $('.toast').toast('show');

        setTimeout(() => {
            toast.remove();
        }, 3500);
    }
  </script>
</body>
</html>
```

Output:

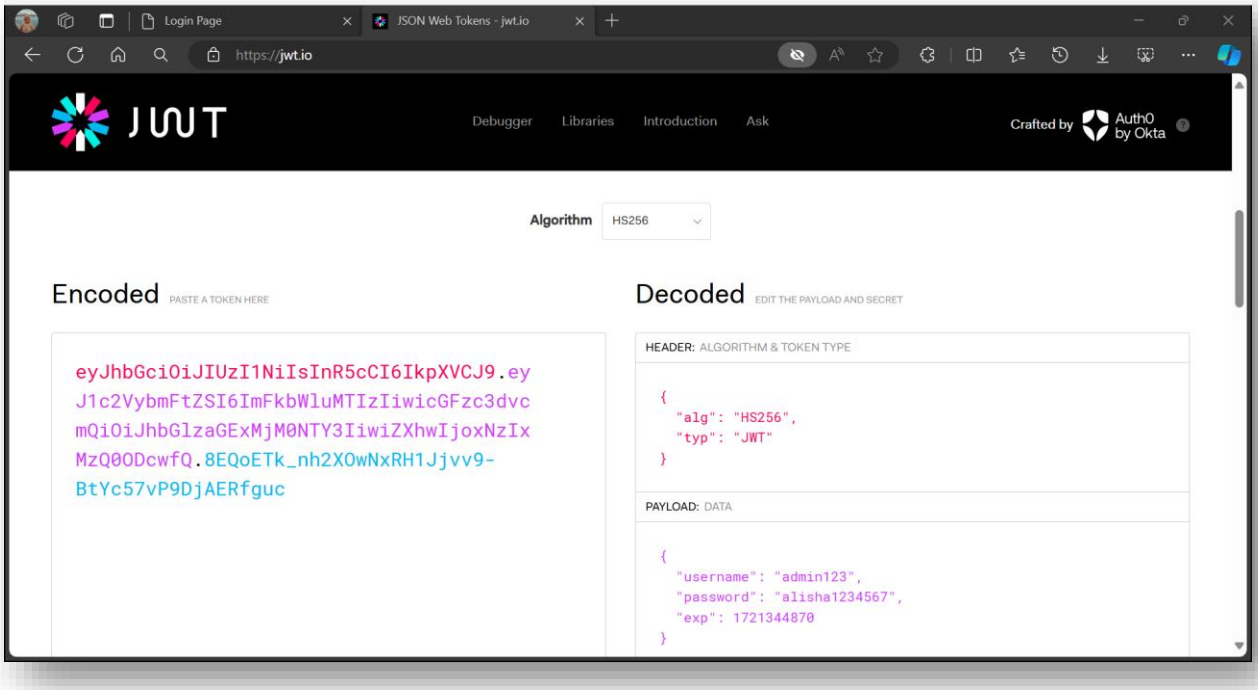
username == 'admin123'

password == 'alisha1234567'





Also can use the website https://jwt.io/ for the decoding of the jwt token:



Secure Login App Code:

.\venv\Scripts\Activate
source venv/bin/activate
pip install Flask PyJWT flask_sqlalchemy
pip install cryptography
pip list

App.py:

```
from flask import Flask, request, jsonify, send_from_directory, make_response
import jwt
import datetime
import os
from werkzeug.security import check_password_hash, generate_password_hash

app = Flask(__name__)

# Secret key for JWT encoding/decoding (strong and securely stored)
SECRET_KEY = os.environ.get('SECRET_KEY', 'my_super_secret_key')

# Example user data for demonstration
USER_DATA = {
    'admin': generate_password_hash('password')
```

```
}

def generate_jwt(username):
    """
    Generates a JWT token with the username in the payload.
    """
    payload = {
        'username': username,
        'exp': datetime.datetime.utcnow() + datetime.timedelta(minutes=15)  # Short expiration time
    }
    token = jwt.encode(payload, SECRET_KEY, algorithm='HS256')
    return token

def generate_refresh_token(username):
    """
    Generates a refresh token with the username in the payload.
    """
    payload = {
        'username': username,
        'exp': datetime.datetime.utcnow() + datetime.timedelta(days=7)  # Longer expiration for refresh
token
    }
    token = jwt.encode(payload, SECRET_KEY, algorithm='HS256')
    return token

@app.route('/')
def index():
    """
    Serves the login HTML page.
    """
    return send_from_directory(os.path.join(app.root_path, 'static'), 'index.html')

@app.route('/login', methods=['POST'])
def login():
    """
    Handles login requests. Validates credentials and returns a JWT token if valid.
    """
    data = request.get_json()
    if not data or 'username' not in data or 'password' not in data:
        return jsonify({'message': 'Bad Request'}), 400

    username = data['username']
    password = data['password']

    # Validate credentials
    if username in USER_DATA and check_password_hash(USER_DATA[username], password):
        token = generate_jwt(username)
        refresh_token = generate_refresh_token(username)
```

```
        response = make_response(jsonify({'message': 'Login successful'}))
        response.set_cookie('token', token, httponly=True, secure=True)  # Secure HTTP-only cookie
        response.set_cookie('refresh_token', refresh_token, httponly=True, secure=True)  # Secure HTTP-
only cookie

        return response
    else:
        return jsonify({'message': 'Invalid credentials'}), 401

@app.route('/refresh', methods=['POST'])
def refresh():
    """
    Handles token refresh requests. Validates the refresh token and returns a new JWT token if valid.
    """
    refresh_token = request.cookies.get('refresh_token')
    if not refresh_token:
        return jsonify({'message': 'No refresh token provided'}), 400

    try:
        decoded = jwt.decode(refresh_token, SECRET_KEY, algorithms=['HS256'])
        username = decoded['username']
        token = generate_jwt(username)

        response = make_response(jsonify({'message': 'Token refreshed'}))
        response.set_cookie('token', token, httponly=True, secure=True)

        return response
    except jwt.ExpiredSignatureError:
        return jsonify({'message': 'Refresh token expired'}), 401
    except jwt.InvalidTokenError:
        return jsonify({'message': 'Invalid refresh token'}), 401

@app.route('/favicon.ico')
def favicon():
    """
    Serves a placeholder favicon to avoid 404 error.
    """
    return send_from_directory(os.path.join(app.root_path, 'static'), 'favicon.ico')

if __name__ == '__main__':
    # Running the Flask application in debug mode
    app.run(debug=True)
```

Static (folder):

Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Login Page</title>
   <link href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"
rel="stylesheet">
   <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.16.0/umd/popper.min.js"></script>
   <script src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></script>
   <style>
      .token-box {
         word-wrap: break-word;
         white-space: pre-wrap;
         background-color: #d4edda;
         border-color: #c3e6cb;
         color: #155724;
         padding: 10px;
         border-radius: 5px;
         margin-top: 10px;
         position: relative;
      }
      .spinner-border {
         display: none;
      }
      .toast {
         position: fixed;
         top: 20px;
         right: 20px;
         z-index: 1050;
      }
      .jwt-info {
         margin-top: 10px;
         display: none;
      }
      .decoded-step {
         margin-top: 10px;
         border: 1px solid #c3e6cb;
         border-radius: 5px;
         padding: 10px;
         background-color: #f8f9fa;
      }
   </style>
</head>
<body>
```

```
  <div class="container">
    <div class="row justify-content-center">
      <div class="col-md-6">
        <h2 class="text-center mt-5">Login Page</h2>
        <form id="loginForm" class="mt-3 needs-validation" novalidate>
          <div class="form-group">
            <label for="username">Username:</label>
            <input type="text" id="username" name="username" class="form-control" required>
            <div class="invalid-feedback">
              Please enter your username.
            </div>
          </div>
          <div class="form-group">
            <label for="password">Password:</label>
            <input type="password" id="password" name="password" class="form-control"
required>
            <div class="invalid-feedback">
              Please enter your password.
            </div>
          </div>
          <button type="submit" class="btn btn-primary btn-block">
            Login
            <span class="spinner-border spinner-border-sm" role="status" aria-
hidden="true"></span>
          </button>
        </form>
        <div id="response" class="mt-3"></div>
      </div>
    </div>
  </div>

  <div id="toast-container"></div>

  <script>
    (function() {
      'use strict';
      window.addEventListener('load', function() {
        var forms = document.getElementsByClassName('needs-validation');
        Array.prototype.filter.call(forms, function(form) {
          form.addEventListener('submit', function(event) {
            if (form.checkValidity() === false) {
              event.preventDefault();
              event.stopPropagation();
            }
            form.classList.add('was-validated');
          }, false);
        });
      }, false);
```

```
})();

document.getElementById('loginForm').addEventListener('submit', async function(event) {
    event.preventDefault();

    const form = event.target;
    if (form.checkValidity() === false) {
        event.stopPropagation();
        form.classList.add('was-validated');
        return;
    }
    form.classList.remove('was-validated');

    const spinner = document.querySelector('.spinner-border');
    spinner.style.display = 'inline-block';

    const username = document.getElementById('username').value;
    const password = document.getElementById('password').value;

    const response = await fetch('/login', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({ username, password })
    });

    const result = await response.json();

    spinner.style.display = 'none';

    const responseDiv = document.getElementById('response');
    if (response.ok) {
        showToast('Login successful! JWT token generated and stored securely.', 'success');
    } else {
        responseDiv.innerHTML = '<div class="alert alert-danger">Error: ' + result.message + '</div>';
        showToast('Login failed. Please check your credentials.', 'danger');
    }
});

function showToast(message, type) {
    const toastContainer = document.getElementById('toast-container');
    const toast = document.createElement('div');
    toast.className = `toast align-items-center text-white bg-${type} border-0`;
    toast.setAttribute('role', 'alert');
    toast.setAttribute('aria-live', 'assertive');
    toast.setAttribute('aria-atomic', 'true');
```

```
        toast.innerHTML = `
          <div class="d-flex">
            <div class="toast-body">
              ${message}
            </div>
            <button type="button" class="btn-close btn-close-white me-2 m-auto" data-bs-
dismiss="toast" aria-label="Close"></button>
          </div>
        `;

        toastContainer.appendChild(toast);
        const bsToast = new bootstrap.Toast(toast);
        bsToast.show();
      }
    </script>
</body>
</html>
```

Explanation of Improvements

1. **Avoid Including Sensitive Information**:
    - Sensitive information such as passwords are not included in the JWT payload.
2. **Strong Secret Keys**:
    - The SECRET_KEY is stored securely and used to sign the JWTs. A strong key is essential for security.
3. **Short Expiration Time**:
    - JWT tokens are set to expire in 15 minutes, minimizing the risk if a token is compromised.
4. **HTTPS Transmission**:
    - The set_cookie method sets cookies with the secure=True flag, ensuring they are only sent over HTTPS.
5. **HTTP-Only Cookies**:
    - Tokens are stored in HTTP-only cookies to prevent access from JavaScript, reducing the risk of XSS attacks.
6. **Refresh Tokens**:
    - Implemented refresh tokens with a longer expiration period to allow users to maintain their session without repeatedly logging in.

Output: